

LLMsVerifier

LLMsVerifier

Doğrula. İzle. Optimize Et.

Özet

LLMsVerifier, birden fazla sağlayıcıdaki Büyük Dil Modelleri'ni doğrulamak, izlemek ve optimize etmek için kurumsal düzeyde bir platformdur. Platform, yalnızca gerçekten çalıştığı kanıtlanan modellerin kullanılabilir veya dışa aktarılabilir olarak işaretlenmesini sağlayan zorunlu bir “Kodumu görüyor musun?” doğrulama testine dayanır.

Kısa açıklama

Birden fazla sağlayıcıdaki BDM'leri doğrulayan, kıyaslayan, izleyen ve optimize eden bir Go platformu. Her model, kullanıma sunulmadan önce zorunlu bir kod görünürlüğü testinden geçmek zorundadır; ardından gecikme süresi, akış, fonksiyon çağırma, görüntü işleme ve gömme kontrolleri yapılır ve yalnızca doğrulanmış yapılandırmalar AI CLI araçları için dışa aktarılır.

Uzun açıklama

LLMsVerifier, birden fazla sağlayıcıdaki LLM performansını doğrulamak, izlemek ve optimize etmek için kapsamlı bir platformdur. Temel ilkesi *zorunlu doğrulamadır* ve bu konuda tavizsizdir: Herhangi bir model kullanılabilir olarak işaretlenmeden —ya da dışa aktarılan bir yapılandırmaya dahil edilmeden— önce, sağlayıcıya gerçek HTTP çağrılarını yaparak yanıtı gerçek anlamda kavrayıp kavramadığını (sanki makul görünen bir yankıdan ibaret olmadığını) analiz eden bir “Kodumu görüyor musun?” testinden geçmelidir. Girdinizi açıkça göremediğini ve anlayamadığını kanıtlayan bir model, asla “kullanılabilir” etiketini alamaz. Bu geçitten sonra Doğrulayıcı Motoru, varlık, yanıt verme hızı, gecikme süresi, akış, fonksiyon

çağırma, görüntü işleme ve embeddings gibi yetenek testlerinden oluşan tam bir batarya çalıştırır. Raporlayıcı Motoru ise sonuçları eyleme geçirilebilir markdown ve JSON raporlarına dönüştürür.

Sistem modüler ve olay odaklıdır; CLI, TUI, Web ve REST API arayüzlerini, Doğrulayıcı Motoru, Raporlayıcı Motoru ve Yapılandırma Yöneticisi'nden oluşan bir çekirdek üzerinden sunar ve doğrulama ile sınırlı kalmaz. Gelişmiş katmanlar, LLM destekli görev ayrıştırma için Süpervizör/İşçi modeli, çok uzun oturumların kesintiye uğramamasını sağlayan kayan pencere ve LLM özetleme tabanlı bağlam yönetimi, bulut destekli kontrol noktaları ve devre kesiciler ile gecikme süresine dayalı yönlendirme içeren bir yedekleme sistemi ekler. Çevresel altyapı üretim ortamına uygun şekilde tasarlanmıştır: pub/sub olay veri yolu, cron zamanlama, fiyatlandırma/sınır tespiti, RAG için bir vector veritabanı ve bir dışa aktarma sistemi. Her oluşturulan sağlayıcı/modelin sonuna eklenen (*llmsvd*) imza kuralı sayesinde, doğrulanmış bir çıktı tek bakışta izlenebilir ve asla doğrulanmamış bir modelle karıştırılamaz — ayrıca yalnızca doğrulanmış modeller, OpenCode, Crush ve Claude Code gibi AI CLI araçları için dışa aktarılan yapılandırmalara yazılır. Platform, ekiplerin üretim ortamında gerçekten ihtiyaç duyduğu operasyonel araçlarla birlikte sunulur: Docker/Kubernetes/Helm dağıtımı, Prometheus/Grafana izleme, LDAP/SSO ve SQLCipher şifreli depolama.

Neden geliştirdik?

Çünkü yalnızca yapılandırma tabanlı kontroller güvenilir değildir — bir API anahtarı süresi dolabilir, bir model kullanımdan kaldırılabilir ve bir yapılandırma dosyası, gerçek gecikme süresi, gerçek hatalar ya da modelin girdinizi gerçekten görüp anlaması hakkında hiçbir bilgi vermez. LLMsVerifier, “Yapılandırmada yer alıyorsa çalışıyor olmalı” yaklaşımının yerine kanıtı koyar: Yalnızca doğru yanıt verdiği kanıtlanan modeller kullanılabilir olarak işaretlenir ve dışa aktarılır.

İçerik

Neden bir oyun değiştirici?

LLM filolarını *güvenilir* hale getiriyor — bu kelime, eksikliklerle yalan söyleyen konfigürasyonların hakim olduğu bir alanda nadiren kazanılır. Yapılandırılmış bir modelin çalışmasını ummak yerine, ekipler her oyundaki her modelin gerçek doğrulama sürecinden geçtiğine dair zorunlu, test edilebilir bir güvenceye sahip oluyor; izleme, yedekleme ve yalnızca doğrulanmış modellerin dışa aktarımı, kanıttan üretime kadar olan döngüyü tamamlıyor. Helix ekosistemi içinde, LLM modeli, sağlayıcı ve doğrulama meta verileri için tek gerçek kaynağı haline geliyor:

Diğer hizmetler (aralarında HelixTranslate da bulunuyor) bu kaynağa yönlendirme yapıyor, böylece tüm platform, “Şu anda hangi modeller gerçekten çalışıyor?” sorusuna tek bir dürüst yanıt alıyor; her ekip kendi umutlu tahminini sürdürmek yerine.

Yenilikçi olan ne?

- **Zorunlu “Kodumu görüyor musun?” doğrulaması** — Bir modelin kullanılabilir hale gelmeden önce geçmesi gereken, gerçek, HTTP destekli bir anlama kapısı; ürünün imza niteliğindeki farkı ve kanıtlanmamış hiçbir şeyin sızdırmamasının nedeni.
- **Yalnızca doğrulanmış konfigürasyon dışı aktarımı** — AI CLI araçları için oluşturulan konfigürasyonlar *yalnızca* doğrulama sürecini geçmiş modelleri içeriyor; böylece gönderdiğiniz konfigürasyon, sessizce bozuk bir modeli yeniden devreye sokamaz.
- **(LLMSVD) marka-soneki sistemi** — Oluşturulan her sağlayıcı/model, çıktının gittiği her yerde izlenebilir bir sonek taşıyor; doğrulanmış kökenin her yerde görünür olmasını sağlıyor.
- **Birçok CLI aracısı ve sağlayıcıda yetenek tespiti** — Akış türlerini (SSE, WebSocket, JSONL, EventStream), sıkıştırma ve önbellekleme davranışlarını varsaymak yerine parmak izi alıyor.
- **Dayanıklı yedekleme** — Devre kesiciler, ilk-token süresi belirli bir eşiği aştığında yeniden yönlendirme yapan gecikme tabanlı yönlendirme, sağlık kontrolleri ve ağırlıklı trafik bölüşümü, bireysel sağlayıcılar sallandığında filonun yanıt vermeye devam etmesini sağlıyor.
- **Uzun süreli özerklik** — Süpervizör/İşçi ayrıştırma modeli, büyük işleri yönetilebilir parçalara bölüyor; kesintilerde ilerlemenin kaybolmaması için bulut depolamaya periyodik kontrol noktaları ekleniyor; modelin bağlamda boğulmadan konuyu sürdürebilmesi için katmanlı bağlam yönetimi (kayan pencere + LLM özetleme + RAG) kullanılıyor.
- **RAG / vector-DB entegrasyonu** ile bağlamsal zenginleştirme.

En büyük teknik zorluklar ve çözümlerimiz

- **Bir modelin yalnızca yapılandırılmış değil, gerçekten çalıştığını kanıtlama.** Asıl amaç ve en zor kısım. Gerçek API çağrılarını yapan ve yanıtları olumlu anlamıyla analiz eden zorunlu kod görünürlüğü testi ile çözüldü; geniş bir yetenek testi paketiyle destekleniyor — ardından doğrulama sürecini geçmeyen hiçbir şey dışı aktarılmıyor; böylece üretimi sınırlayan şey konfigürasyon değil, kanıt oluyor.

- **Birçok güvenilirmez üçüncü taraf sağlayıcıda güvenilirlik.** Sağlayıcı istikrarsızlığını normal bir durum olarak ele alan bir yedekleme orkestratörü ile çözüldü: Devre kesiciler, N başarısızlık sonrası M saniye içinde bir sağlayıcıyı zayıflamış olarak işaretliyor; gecikme tabanlı yönlendirme, yavaş uç noktalardan uzaklaştırıyor; periyodik sağlık kontrolleri iyileşmeyi tespit ediyor; ağırlıklı yönlendirme, maliyet etkin modellerle premium modeller arasında denge kuruyor.
- **Çok uzun, özerk oturumları sürdürme.** Büyük işleri yönetilebilir parçalara bölen Süpervizör/İşçi ayrıştırma modeli ile çözüldü; kesintilerde ilerlemenin kaybolmaması için bulut depolamaya periyodik kontrol noktaları ekleniyor; modelin bağlamda boğulmadan konuyu sürdürebilmesi için katmanlı bağlam yönetimi (kayan pencere + LLM özetleme + RAG) kullanılıyor.
- **Sağlayıcı çeşitliliği.** Birçok sağlayıcıya özgü Go adaptörünü tek bir ortak arayüzün arkasına gizleyerek çözüldü; gerçek uç noktalar merkezi olarak listeleniyor — böylece bir sağlayıcı eklemek, kod tabanında dalgalanma yaratmak yerine sınırlı bir değişiklik haline geliyor.

Teknoloji Yığını

- **Go** — eşzamanlılık özelliği nedeniyle çekirdek platform dili olarak seçildi; çok iş parçacıklı Doğrulayıcı Motoru'nu çalıştırarak birçok modeli paralel olarak sorgulayabilen ve çevresindeki hizmetleri destekleyen bir yapı sunuyor.
- **Gin** — REST API sunucusu olarak seçildi; JWT kimlik doğrulama, hız sınırlama ve WebSocket/SSE uç noktalarını barındırıyor.
- **SQLite + SQLCipher** — gömülü depolama için seçildi; veritabanı düzeyinde şifreleme sunuyor çünkü doğrulama verileri (anahtarlar, sonuçlar) hassas nitelikte ve varsayılan olarak beklemede şifrelenmeli.
- **Redis** — sıcak doğrulama ve meta veri sorgularını hızlı tutmak için önbellek katmanı olarak seçildi.
- **RabbitMQ + Kafka** — olay odaklı mimariyi güçlendirmek için seçildi; platform genelinde üreticileri tüketicilerden ayıran mesajlaşma ve akış sistemlerini sağlıyor.
- **gRPC + Protocol Buffers** — bileşenler arası güçlü tür denetimine sahip iletişim ve olay taşımacılığı için seçildi.
- **QUIC / HTTP-3 (quic-go)** — modern taşıma protokolü desteği için seçildi (belgelerde HTTP/3 sağlayıcı desteğinin sınırlı olduğu belirtiliyor — sunulan bir yetenek, evrensel bir iddia değil).
- **JWT + LDAP/NTLM** — kurumsal kimlik doğrulama için seçildi; platformun mevcut kurumsal kimlik altyapısına (SSO/ SAML/OIDC) entegre olmasını sağlıyor (belgelerde iddia edildiği üzere).

- **Viper (yapılandırma), Logrus (kayıt), Brotli/sıkıştırma (sıkıştırma)** — operasyonel altyapı: esnek yapılandırma, yapılandırılmış kayıtlar ve yük sıkıştırma.
- **Angular** — doğrulama ve izleme için görsel arayüz olan tek sayfalık web uygulaması için seçildi.
- **Python + JavaScript SDK'ları** — istemci ekiplerine birinci sınıf erişim imkânı sunmak için seçildi; OpenAPI/Swagger aracılığıyla belgeleniyor.
- **Docker, Kubernetes, Helm** — üretim ortamına dağıtım, sağlık izleme ve otomatik ölçeklendirme için seçildi; böylece doğrulama filosu modern bir hizmet gibi ölçeklenebiliyor.
- **Prometheus + Grafana** — metrikler ve kontrol panelleri için seçildi; platformun kendi sağlığını izlediği modeller kadar gözlemlenebilir kılıyor.
- **Testify (Go) + node -test/jsdom (web)** — Go çekirdeği ve web ön yüzü genelinde katmanlı testler için seçildi.

Durum ve Dürüstlük Notları

- **Durum: beta.** Go kaynağı gerçek HTTP doğrulamasını uyguluyor (doğrulamanın yalnızca yapılandırma tabanlı olduğunu anlatan eski bir belge ise hayal ürünü ve güncelliğini yitirmiş durumda — yetkili kaynak koddur).
- **Lisans: belirsiz.** README dosyası MIT lisansını belirtirken, bir Dockerfile etiketi Apache-2.0 diyor — yayımlanmadan önce netleştirilmeli.
- **Sağlayıcı sayısı:** README dosyası “12 adaptör” derken, sağlayıcılar dizini yaklaşık 26 adet listelemekte — “12+ / devam ediyor” olarak değerlendirilmeli. Çok sayıda “SON/TAMLANMIŞ” ibareli hevesli durum dosyası mevcut; yetkili kaynak kod, belgeler ve go.mod dosyasıdır.
- Depo vasic-digital organizasyonunda yer alıyor, ancak işlevsel olarak Helix LLM-altyapı kümesinin güven katmanını oluşturuyor.

Öncelik seviyesi: Helix-birincil (LLM-altyapı kümesi; LLM/sağlayıcı/doğrulama meta verileri için tek gerçek kaynak). HelixTrack’dan sonra gelir.